

Graphs, Prime Numbers, and Complexity

Graphs

A graph is used in the project in the `SimilarItemsGraph` class to get the most similar items to a particular item. This graph is implemented as an undirected, weighted graph, as items can be more similar to one item than another, and the similarity symmetric. When an item is searched, the graph runs Dijkstra's algorithm to get the 10 most similar items to the searched item. The graph is filled at launch by calculating the similarity between each pair of items and adding the edge if it is in the top 5% of edge weights among all possible edges. Citation is in the file.

- Determining the limit
 - A script was run in `similarityGraphing.ipynb` once all possible edges were written out by `SimilarityAnalysis` to a `.dat` file. The script then found the 95th percentile of edge weights, which is now hardcoded into the graph class to limit the number of edges to improve algorithm run time.
- Representation
 - The graph is represented as an adjacency list, as it is more space efficient than an adjacency matrix for sparse graphs, which is the case here as only the top 5% of edges are added to the graph.
- Why it was chosen
 - As shown in the documentation for `Algorithm Design Paradigms`, the greedy nature of Dijkstra's algorithm is desirable for our case, as paths through more nodes are punished.
 - The algorithm is well-known, and simple to implement.

Prime Numbers

A prime number is used in the project in the `IdHashKey` class to implement the Rabin-Karp rolling hash function for the custom hash function. The prime number is used to prevent overflow issues, as well as prevent collisions. The prime number should be large, so it was chosen as the largest prime number that is under 100 million.

Complexity

- **P**: Most algorithms used are **P**, as they are solved in the worst case $O(n^2)$ time, which is a polynomial time.
- **NP**: Since most algorithms used are **P**, they are also **NP**, as $P \subseteq NP$.
- **NP-Complete**: No algorithms used are **NP**-complete, as they are all solvable in polynomial time.
- **NP-Hard**: No algorithms used are **NP**-hard, as they are all solvable in polynomial time.